

chrome.identity

Use the `chrome.identity` API to get OAuth2 access tokens.

Permissions

`identity`

Types

AccountInfo

PROPERTIES

`id`

`string`



A unique identifier for the account. This ID will not change for the lifetime of the account.

AccountStatus

[Chrome 84+](#)

ENUM

`"SYNC"`

Specifies that Sync is enabled for the primary account.

`"ANY"`

Specifies the existence of a primary account, if any.

GetAuthTokenResult

Chrome 105+

PROPERTIES

grantedScopes

string[] **optional**

A list of OAuth2 scopes granted to the extension.

token

string **optional**

The specific token associated with the request.

InvalidTokenDetails

PROPERTIES

token

string

The specific token that should be removed from the cache.

ProfileDetails

Chrome 84+

PROPERTIES

accountStatus

AccountStatus (#type=AccountStatus) **optional**

A status of the primary account signed into a profile whose `ProfileUserInfo` should be returned. Defaults to SYNC account status.

ProfileUserInfo

PROPERTIES

email

string

An email address for the user account signed into the current profile. Empty if the user is not signed in or the `identity.email` manifest permission is not specified.

id

string

A unique identifier for the account. This ID will not change for the lifetime of the account. Empty if the user is not signed in or (in M41+) the `identity.email` manifest permission is not specified.

TokenDetails

PROPERTIES

account

AccountInfo (#type=AccountInfo) **optional**

The account ID whose token should be returned. If not specified, the function will use an account from the Chrome profile: the Sync account if there is one, or otherwise the first Google web account.

enableGranularPermissions

boolean **optional**

Chrome 87+

The `enableGranularPermissions` flag allows extensions to opt-in early to the granular permissions consent screen, in which requested permissions are granted or denied

individually.

interactive

boolean **optional**

Fetching a token may require the user to sign-in to Chrome, or approve the application's requested scopes. If the `interactive` flag is `true`, `getAuthToken` will prompt the user as necessary. When the flag is `false` or omitted, `getAuthToken` will return failure any time a prompt would be required.

scopes

string[] **optional**

A list of OAuth2 scopes to request.

When the `scopes` field is present, it overrides the list of scopes specified in `manifest.json`.

WebAuthFlowDetails

PROPERTIES

abortOnLoadForNonInteractive

boolean **optional**

[Chrome 113+](#)

Whether to terminate `launchWebAuthFlow` for non-interactive requests after the page loads. This parameter does not affect interactive flows.

When set to `true` (default) the flow will terminate immediately after the page loads. When set to `false`, the flow will only terminate after the `timeoutMsForNonInteractive` passes. This is useful for identity providers that use JavaScript to perform redirections after the page loads.

interactive

boolean **optional**

Whether to launch auth flow in interactive mode.

Since some auth flows may immediately redirect to a result URL, `launchWebAuthFlow` hides its web view until the first navigation either redirects to the final URL, or finishes loading a page meant to be displayed.

If the `interactive` flag is `true`, the window will be displayed when a page load completes. If the flag is `false` or omitted, `launchWebAuthFlow` will return with an error if the initial navigation does not complete the flow.

For flows that use JavaScript for redirection, `abortOnLoadForNonInteractive` can be set to `false` in combination with setting `timeoutMsForNonInteractive` to give the page a chance to perform any redirects.

`timeoutMsForNonInteractive`

number **optional**

Chrome 113+

The maximum amount of time, in milliseconds, `launchWebAuthFlow` is allowed to run in non-interactive mode in total. Only has an effect if `interactive` is `false`.

`url`

string

The URL that initiates the auth flow.

Methods

`clearAllCachedAuthTokens()`

Chrome 87+

```
chrome.identity.clearAllCachedAuthTokens(): Promise<void>
```

Resets the state of the Identity API:

- Removes all OAuth2 access tokens from the token cache
- Removes user's account preferences
- De-authorizes the user from all auth flows

RETURNS

Promise<void>

Chrome 106+

getAccounts()

Dev channel

```
chrome.identity.getAccounts(): Promise<AccountInfo (#type=AccountInfo)[ ]>
```

Retrieves a list of AccountInfo objects describing the accounts present on the profile.

`getAccounts` is only supported on dev channel.

RETURNS

Promise<AccountInfo (#type=AccountInfo)[]>

getAuthToken()

```
chrome.identity.getAuthToken(  
  details?: TokenDetails (#type-TokenDetails),  
) : Promise<GetAuthTokenResult (#type-GetAuthTokenResult)>
```

Gets an OAuth2 access token using the client ID and scopes specified in the oauth2 section of manifest.json (https://developer.chrome.com/docs/apps/app_identity#update_manifest).

The Identity API caches access tokens in memory, so it's ok to call `getAuthToken` non-interactively any time a token is required. The token cache automatically handles expiration.

For a good user experience it is important interactive token requests are initiated by UI in your app explaining what the authorization is for. Failing to do this will cause your users to get authorization requests, or Chrome sign in screens if they are not signed in, with with no context. In particular, do not use `getAuthToken` interactively when your app is first launched.

Note: When called with a callback, instead of returning an object this function will return the two properties as separate arguments passed to the callback.

PARAMETERS

details

TokenDetails (#type-TokenDetails) **optional**

Token options.

RETURNS

Promise<GetAuthTokenResult (#type-GetAuthTokenResult)>

Chrome 105+

`getProfileUserInfo()`

```
chrome.identity.getProfileUserInfo(  
  details?: ProfileDetails (#type-ProfileDetails),  
) : Promise<ProfileUserInfo (#type-ProfileUserInfo)>
```

Retrieves email address and obfuscated gaia id of the user signed into a profile.

Requires the `identity.email` manifest permission. Otherwise, returns an empty result.

This API is different from `identity.getAccounts` in two ways. The information returned is available offline, and it only applies to the primary account for the profile.

PARAMETERS

details

ProfileDetails (#type-ProfileDetails) **optional**

Chrome 84+

Profile options.

RETURNS

Promise<ProfileUserInfo (#type-ProfileUserInfo)>

Chrome 106+

getRedirectURL()

```
chrome.identity.getRedirectURL(  
  path?: string,  
) : string
```

Generates a redirect URL to be used in `launchWebAuthFlow`.

The generated URLs match the pattern `https://<app-id>.chromiumapp.org/*`.

PARAMETERS

path

string **optional**

The path appended to the end of the generated URL.

RETURNS

string

launchWebAuthFlow()

```
chrome.identity.launchWebAuthFlow(  
  details: WebAuthFlowDetails (#type-WebAuthFlowDetails),  
): Promise<string | undefined>
```

Starts an auth flow at the specified URL.

This method enables auth flows with non-Google identity providers by launching a web view and navigating it to the first URL in the provider's auth flow. When the provider redirects to a URL matching the pattern `https://<app-id>.chromiumapp.org/*`, the window will close, and the final redirect URL will be passed to the `callback` function.

For a good user experience it is important interactive auth flows are initiated by UI in your app explaining what the authorization is for. Failing to do this will cause your users to get authorization requests with no context. In particular, do not launch an interactive auth flow when your app is first launched.

PARAMETERS

details

WebAuthFlowDetails (#type-WebAuthFlowDetails)

WebAuth flow options.

RETURNS

Promise<string | undefined>

Chrome 106+

removeCachedAuthToken()

```
chrome.identity.removeCachedAuthToken(  
  details: InvalidTokenDetails (#type-InvalidTokenDetails),  
): Promise<void>
```

Removes an OAuth2 access token from the Identity API's token cache.

If an access token is discovered to be invalid, it should be passed to `removeCachedAuthToken` to remove it from the cache. The app may then retrieve a fresh token with `getAuthToken`.

PARAMETERS

details

InvalidTokenDetails (#type-InvalidTokenDetails)

Token information.

RETURNS

Promise<void>

Chrome 106+

Events

onSignInChanged

```
chrome.identity.onSignInChanged.addListener(  
  callback: function,  
)
```

Fired when signin state changes for an account on the user's profile.

PARAMETERS

callback

function

The `callback` parameter looks like:

```
(account: AccountInfo (#type=AccountInfo), signIn: boolean) => void
```

account

AccountInfo (#type=AccountInfo)

signIn

boolean

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-08-11 UTC.